

IIT Jammu Hostel Allocation System: Database Design Report

Executive Summary

This report presents a comprehensive database design for the IIT Jammu Hostel Allocation System. The system manages the allocation of hostel accommodations to students, tracking student information, hostel details, room assignments, batch management, roommate pairings, and room allocation zones. The final database schema consists of seven normalized tables (HOSTEL, ADMINISTRATOR, BATCHES, STUDENT, ROOM, ROOMMATES, and ROOM_ALLOC_ZONES), all designed to conform to Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF) standards. This design ensures data integrity, eliminates redundancy, and provides efficient query performance for hostel allocation operations. The report includes an Entity-Relationship diagram to provide comprehensive visualization of the database structure from a semantic and theoretical perspective.

1. Database Overview

1.1 System Purpose

The IIT Jammu Hostel Allocation System database is designed to support institutional hostel management operations.

The primary objectives include:

- Student Information Management:** Maintain comprehensive records of students including academic details, contact information, and batch assignments
- Hostel Repository:** Store information about available hostels, their capacity, and occupancy requirements
- Administrator Management:** Track administrators responsible for hostel management
- Batch Management:** Organize students into batches by program and year of study
- Room Management:** Track individual rooms within hostels with detailed location information
- Roommate Association:** Manage roommate pairings and group assignments for students
- Zone-Based Allocation:** Define room allocation zones for systematic batch-wise room assignment

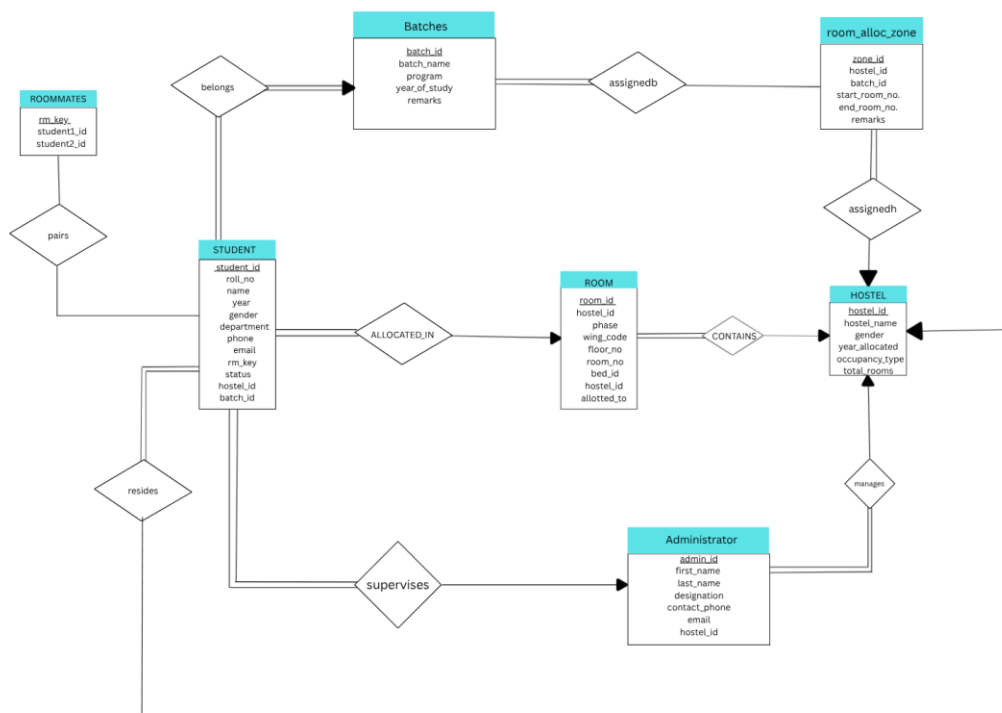
1.2 System Scope

The database encompasses the complete lifecycle of hostel allocation, from student registration through room assignment, batch organization, and roommate pairing. The system supports both single and double occupancy rooms and allows for flexible management of student status (Active, Graduated, Left). It also provides administrators with the

2. Entity-Relationship Diagram

2.1 E-R Diagram

The following diagram presents the database structure showing all entities and their relationships:



The E-R diagram illustrates the following key entities and relationships:

Entities:

- HOSTEL
- ADMINISTRATOR
- BATCHES
- STUDENT
- ROOM
- ROOMMATES
- ROOM_ALLOC_ZONES

Key Relationships:

1. **MANAGES** (ADMINISTRATOR ↔ HOSTEL): One administrator manages one hostel
2. **RESIDES** (STUDENT ↔ HOSTEL): Many students reside in one hostel
3. **BELONGS** (STUDENT ↔ BATCHES): Many students belong to one batch
4. **ALLOCATED_IN** (STUDENT ↔ ROOM): One student is allocated to at most one room bed
5. **CONTAINS** (HOSTEL ↔ ROOM): One hostel contains many rooms

6. **PAIRS** (STUDENT ↔ ROOMMATES): Students are paired in roommate groups

7. **ASSIGNEDB** (ROOM_ALLOC_ZONES ↔ BATCHES and HOSTEL): Zones are assigned for batch-hostel combinations

2.2 Detailed Relationship Explanations

1. MANAGES Relationship (ADMINISTRATOR ↔ HOSTEL)

- **Cardinality:** One-to-Many (1:M) - Multiple administrators can manage one hostel
- **Semantics:** Each administrator is assigned to exactly one hostel
- **Type:** Mandatory on both sides
- **Example:** Administrator "Dr. Kumar" manages Hostel "Sarai"
- **Query Implication:** "Get all administrators for Hostel X" uses this relationship

2. RESIDES Relationship (STUDENT ↔ HOSTEL)

- **Cardinality:** Many-to-One (M:1) - Many students reside in one hostel
- **Semantics:** Each student must be assigned to exactly one hostel
- **Type:** Mandatory participation
- **Example:** 50 students reside in Hostel "Sarai"
- **Query Implication:** "Get all students in Hostel X" uses this relationship

3. BELONGS Relationship (STUDENT ↔ BATCHES)

- **Cardinality:** Many-to-One (M:1) - Many students belong to one batch
- **Semantics:** Each student is part of one batch (or NULL if not assigned)
- **Type:** Optional participation (allows NULL)
- **Example:** 30 students belong to Batch "2024-UG-CSE"
- **Query Implication:** "Get all students in Batch X" uses this relationship

4. ALLOCATED_IN Relationship (STUDENT ↔ ROOM)

- **Cardinality:** One-to-One (1:1 or NULL) - One student is allocated to at most one room bed
- **Semantics:** One room bed can be allocated to at most one student
- **Type:** Partial participation (optional) - a room may be unoccupied, a student may not have allocation
- **Example:** Student with roll_no S001 is allocated to Room 301-A1, Room 301-B1 is unoccupied
- **Query Implication:** "Get the student allocated to Room X" uses this relationship

5. CONTAINS Relationship (HOSTEL ↔ ROOM)

- **Cardinality:** One-to-Many (1:M) - One hostel contains many rooms
- **Semantics:** Each room must belong to exactly one hostel
- **Type:** Mandatory on both sides
- **Example:** Hostel "Sarai" contains 50 rooms
- **Query Implication:** "Get all rooms in Hostel X" uses this relationship

6. PAIRS Relationship (STUDENT ↔ ROOMMATES)

- **Cardinality:** One-to-One (1:1) - One student may be part of exactly one roommate group
- **Semantics:** One roommate group contains one to two students (1:2)
- **Type:** Optional for the second student (single occupancy scenarios)
- **Example:** Roommate group RM001 contains S001 and S002; RM002 contains S003 (single)
- **Query Implication:** "Get all roommates of Student X" uses this relationship

7. ASSIGNEDB Relationship (ROOM_ALLOC_ZONES ↔ BATCHES and HOSTEL)

- **Cardinality:** Many-to-One (M:1) for both BATCHES and HOSTEL
 - **Semantics:** Each zone is defined for one batch in one hostel
 - **Type:** Mandatory participation
 - **Example:** Zone Z001 assigns rooms 101-120 for Batch "2024-UG" in Hostel "Sarai"
 - **Query Implication:** "Get allocation zones for Batch X in Hostel Y" uses this relationship
-

3. Relational Schema Design

3.1 HOSTEL Table

Relation Schema: HOSTEL(hostel_id, hostel_name, gender, year_allocated, occupancy_type, total_rooms)

Description: The HOSTEL table maintains information about all available hostels in the institution. Each hostel is assigned to a specific gender category and academic year.

Attributes:

- **hostel_id** (INT, AUTO_INCREMENT): Unique identifier and primary key for each hostel
- **hostel_name** (VARCHAR(50), UNIQUE): Distinctive name of the hostel (must be unique)
- **gender** (ENUM: 'Male', 'Female', 'Other'): Gender category for which the hostel is designated
- **year_allocated** (INT): Academic year for which the hostel is allocated
- **occupancy_type** (ENUM: 'Single', 'Double'): Standard occupancy configuration for all rooms in the hostel
- **total_rooms** (INT): Total number of rooms available in the hostel

Constraints:

- **Primary Key:** hostel_id
- **Unique Key:** hostel_name
- All attributes are NOT NULL except those explicitly marked as nullable

SQL Schema:

```
CREATE TABLE hostel (  
    hostel_id INT AUTO_INCREMENT PRIMARY KEY,  
    hostel_name VARCHAR(50) UNIQUE NOT NULL,  
    gender ENUM('Male', 'Female', 'Other'),  
    year_allocated INT NOT NULL,  
    occupancy_type ENUM('Single', 'Double') NOT NULL,  
    total_rooms INT  
);
```

3.2 ADMINISTRATOR Table

Relation Schema: ADMINISTRATOR(admin_id, first_name, last_name, designation, contact_phone, email, hostel_id)

Description: The ADMINISTRATOR table stores information about administrators responsible for managing hostels. Multiple administrators can be assigned to the same hostel.

Attributes:

- `admin_id` (INT, AUTO_INCREMENT): Unique identifier and primary key for each administrator
- `first_name` (VARCHAR(50)): First name of the administrator
- `last_name` (VARCHAR(50)): Last name of the administrator
- `designation` (VARCHAR(100)): Job title or designation of the administrator
- `contact_phone` (VARCHAR(15)): Contact phone number
- `email` (VARCHAR(100), UNIQUE): Email address for communication (must be unique)
- `hostel_id` (INT): Foreign key referencing HOSTEL table

Constraints:

- **Primary Key:** `admin_id`
- **Unique Key:** `email`
- **Foreign Key:** `hostel_id` references HOSTEL(`hostel_id`) ON UPDATE CASCADE ON DELETE RESTRICT

SQL Schema:

```
CREATE TABLE administrator (  
    admin_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    designation VARCHAR(100) NOT NULL,  
    contact_phone VARCHAR(15),  
    email VARCHAR(100) UNIQUE,  
    hostel_id INT NOT NULL,  
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)  
        ON UPDATE CASCADE ON DELETE RESTRICT  
);
```

3.3 BATCHES Table

Relation Schema: BATCHES(`batch_id`, `batch_name`, `program`, `year_of_study`, `remarks`)

Description: The BATCHES table organizes students into batches based on their program and year of study.

Attributes:

- `batch_id` (VARCHAR(50)): Unique identifier and primary key for each batch
- `batch_name` (VARCHAR(50), UNIQUE): Distinctive name of the batch (must be unique)
- `program` (ENUM: 'UG', 'MTech', 'PhD', 'Guest', 'Other', DEFAULT 'UG'): Academic program type
- `year_of_study` (INT, NULL): Year of study for the batch
- `remarks` (VARCHAR(100)): Additional notes or comments

Constraints:

- **Primary Key:** batch_id
- **Unique Key:** batch_name

SQL Schema:

```
CREATE TABLE batches (  
    batch_id VARCHAR(50) PRIMARY KEY,  
    batch_name VARCHAR(50) UNIQUE NOT NULL,  
    program ENUM('UG', 'MTech', 'PhD', 'Guest', 'Other') DEFAULT 'UG',  
    year_of_study INT NULL,  
    remarks VARCHAR(100)  
);
```

3.4 STUDENT Table

Relation Schema: STUDENT(student_id, roll_no, name, year, gender, department, phone, email, rm_key, status, hostel_id, batch_id)

Description: The STUDENT table stores comprehensive information about all students in the hostel allocation system. It maintains personal, academic, and contact details along with roommate group assignment and status tracking.

Attributes:

- student_id (INT, AUTO_INCREMENT): Unique identifier and primary key for each student
- roll_no (VARCHAR(15), UNIQUE): Unique roll number assigned by the institution
- name (VARCHAR(100)): Full name of the student
- year (INT): Current academic year of the student
- gender (ENUM: 'Male', 'Female', 'Other'): Gender identity of the student
- department (VARCHAR(50)): Academic department or school of the student
- phone (VARCHAR(15)): Contact phone number
- email (VARCHAR(100)): Email address for communication
- rm_key (INT, DEFAULT 0): Roommate group identifier; 0 indicates no roommate assignment yet
- status (ENUM: 'Active', 'Graduated', 'Left', DEFAULT 'Active'): Current enrollment status
- hostel_id (INT): Foreign key referencing HOSTEL table
- batch_id (VARCHAR(50)): Foreign key referencing BATCHES table

Constraints:

- **Primary Key:** student_id
- **Unique Key:** roll_no
- **Foreign Key:** hostel_id references HOSTEL(hostel_id) ON UPDATE CASCADE ON DELETE CASCADE
- **Foreign Key:** batch_id references BATCHES(batch_id) ON UPDATE CASCADE ON DELETE SET NULL

SQL Schema:

```
CREATE TABLE student (  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    roll_no VARCHAR(15) UNIQUE NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    year INT NOT NULL,  
    gender ENUM('Male', 'Female', 'Other') NOT NULL,  
    department VARCHAR(50),  
    phone VARCHAR(15),  
    email VARCHAR(100),  
    rm_key INT DEFAULT 0,  
    status ENUM('Active', 'Graduated', 'Left') DEFAULT 'Active',  
    hostel_id INT NOT NULL,  
    batch_id VARCHAR(50),  
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)  
        ON UPDATE CASCADE ON DELETE CASCADE,  
    FOREIGN KEY (batch_id) REFERENCES batches(batch_id)  
        ON UPDATE CASCADE ON DELETE SET NULL  
);
```

3.5 ROOM Table

Relation Schema: ROOM(room_id, hostel_id, phase, wing_code, floor_no, room_no, bed_id, allotted_to)

Description: The ROOM table maintains detailed information about individual rooms within hostels. Each room record includes structural location information and occupant assignment. Rooms can have single or double occupancy based on hostel configuration.

Attributes:

- room_id (INT, AUTO_INCREMENT): Unique identifier and primary key for each room
- hostel_id (INT): Foreign key referencing HOSTEL table; identifies the hostel containing this room
- phase (VARCHAR(5)): Building phase indicator (e.g., '1C', '2A'), useful for hostel layout organization
- wing_code (VARCHAR(10)): Wing or block code within the hostel (e.g., 'A3', 'B1')
- floor_no (INT): Floor number on which the room is located
- room_no (VARCHAR(10)): Room number (e.g., '3001', '3002')
- bed_id (ENUM: 'A1', 'B1', DEFAULT 'A1'): Bed identifier within the room for double occupancy configurations
- allotted_to (INT, NULL): Foreign key referencing STUDENT table; identifies the student currently occupying this bed (NULL if unoccupied)

Constraints:

- **Primary Key:** room_id
- **Unique Composite Key:** UNIQUE(hostel_id, room_no, bed_id) - Ensures no duplicate room+bed combination within a hostel
- **Foreign Key:** hostel_id references HOSTEL(hostel_id) ON UPDATE CASCADE ON DELETE CASCADE
- **Foreign Key:** allotted_to references STUDENT(student_id) ON UPDATE CASCADE ON DELETE SET NULL

SQL Schema:

```
CREATE TABLE room (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    hostel_id INT NOT NULL,
    phase VARCHAR(5),
    wing_code VARCHAR(10),
    floor_no INT,
    room_no VARCHAR(10) NOT NULL,
    bed_id ENUM('A1', 'B1') DEFAULT 'A1',
    allotted_to INT NULL,
    UNIQUE(hostel_id, room_no, bed_id),
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (allotted_to) REFERENCES student(student_id)
        ON UPDATE CASCADE ON DELETE SET NULL
);
```

3.6 ROOMMATES Table

Relation Schema: ROOMMATES (rm_key, student1_id, student2_id)

Description: The ROOMMATES table explicitly defines roommate pairings and groupings. It serves as an explicit link table that maps individual students to their assigned roommate groups, facilitating efficient queries for roommate lookups.

Attributes:

- rm_key (INT): Primary key representing a unique roommate pair/group identifier
- student1_id (INT): Foreign key referencing STUDENT table; first student in the roommate pairing
- student2_id (INT, NULL): Foreign key referencing STUDENT table; second student in the pairing (NULL for single occupancy)

Constraints:

- **Primary Key:** rm_key
- **Foreign Key:** student1_id references STUDENT(student_id) ON UPDATE CASCADE ON DELETE CASCADE

- **Foreign Key:** student2_id references STUDENT(student_id) ON UPDATE CASCADE ON DELETE SET NULL

SQL Schema:

```
CREATE TABLE roommates (
    rm_key INT PRIMARY KEY,
    student1_id INT NOT NULL,
    student2_id INT NULL,
    FOREIGN KEY (student1_id) REFERENCES student(student_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (student2_id) REFERENCES student(student_id)
        ON UPDATE CASCADE ON DELETE SET NULL
);
```

3.7 ROOM_ALLOC_ZONES Table

Relation Schema: ROOM_ALLOC_ZONES(zone_id, hostel_id, batch_id, start_room_no, end_room_no, remarks)

Description: The ROOM_ALLOC_ZONES table defines room allocation zones for systematic batch-wise room assignment. Each zone specifies a range of rooms allocated to a specific batch within a hostel.

Attributes:

- zone_id (INT, AUTO_INCREMENT): Unique identifier and primary key for each zone
- hostel_id (INT): Foreign key referencing HOSTEL table
- batch_id (VARCHAR(50)): Foreign key referencing BATCHES table
- start_room_no (VARCHAR(10)): Starting room number of the allocation zone
- end_room_no (VARCHAR(10)): Ending room number of the allocation zone
- remarks (VARCHAR(100)): Additional notes or comments

Constraints:

- **Primary Key:** zone_id
- **Foreign Key:** hostel_id references HOSTEL(hostel_id) ON UPDATE CASCADE ON DELETE CASCADE
- **Foreign Key:** batch_id references BATCHES(batch_id) ON UPDATE CASCADE ON DELETE CASCADE

SQL Schema:

```
CREATE TABLE room_alloc_zones (  
    zone_id INT AUTO_INCREMENT PRIMARY KEY,  
    hostel_id INT NOT NULL,  
    batch_id VARCHAR(50) NOT NULL,  
    start_room_no VARCHAR(10) NOT NULL,  
    end_room_no VARCHAR(10) NOT NULL,  
    remarks VARCHAR(100),  
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)  
        ON UPDATE CASCADE ON DELETE CASCADE,  
    FOREIGN KEY (batch_id) REFERENCES batches(batch_id)  
        ON UPDATE CASCADE ON DELETE CASCADE  
);
```

4. Normalization Analysis

4.1 Normalization Objectives

Database normalization is a systematic approach to organizing data in tables to eliminate redundancy and improve data integrity. The normalization process involves decomposing relations into progressively higher normal forms. This design follows the classical normalization hierarchy: **1NF** → **2NF** → **3NF** → **BCNF**.

4.2 HOSTEL Table Normalization

Analysis:

First Normal Form (1NF): ✓ PASS

- All attributes contain only atomic values
- No repeating groups or arrays are present
- Each cell in the table contains exactly one value
- No horizontal repetition of data
- The table satisfies 1NF

Second Normal Form (2NF): ✓ PASS

- All non-key attributes (hostel_name, gender, year_allocated, occupancy_type, total_rooms) are fully dependent on the complete primary key (hostel_id)
- No partial dependencies exist
- Since hostel_id is a single-attribute primary key, there cannot be partial dependencies
- The table satisfies 2NF

Third Normal Form (3NF): ✓ PASS

Functional Dependencies:

- $\text{hostel_id} \rightarrow \text{hostel_name}$
- $\text{hostel_id} \rightarrow \text{gender}$
- $\text{hostel_id} \rightarrow \text{year_allocated}$
- $\text{hostel_id} \rightarrow \text{occupancy_type}$
- $\text{hostel_id} \rightarrow \text{total_rooms}$
- No non-key attribute depends on another non-key attribute
- All dependencies are on the primary key only
- No transitive dependencies exist ($A \rightarrow B \rightarrow C$ pattern)

Boyce-Codd Normal Form (BCNF): ✓ PASS

- All determinants are candidate keys
- hostel_id is the only determinant and is the primary key
- No anomalies exist

Conclusion: The HOSTEL table is in BCNF and is properly normalized. The design is optimal for this entity with no redundancy.

4.3 ADMINISTRATOR Table Normalization

Analysis:

First Normal Form (1NF): ✓ PASS

- All attributes contain only atomic values
- No multivalued attributes are present
- No arrays, lists, or nested structures exist
- Each attribute cell contains a single scalar value
- The table is in 1NF

Second Normal Form (2NF): ✓ PASS

- All non-key attributes are fully dependent on the primary key (admin_id)
- No partial dependencies exist
- Since admin_id is a single-attribute primary key, all non-key attributes are naturally fully dependent on it
- The table is in 2NF

Third Normal Form (3NF): ✓ PASS

Functional Dependencies:

- `admin_id` $\rightarrow$ `first_name`
- `admin_id` $\rightarrow$ `last_name`
- `admin_id` $\rightarrow$ `designation`
- `admin_id` $\rightarrow$ `contact_phone`
- `admin_id` $\rightarrow$ `email`
- `admin_id` $\rightarrow$ `hostel_id`
- No non-key attribute depends on another non-key attribute
- All dependencies are on the primary key only
- The `hostel_id` attribute serves as a foreign key reference and does not create transitive dependency
- No chain of dependencies ($A \rightarrow B \rightarrow C$) exists where non-key attributes are involved

Boyce-Codd Normal Form (BCNF): ✓ PASS

- All determinants are candidate keys
- `admin_id` and `email` (unique) are candidate keys
- No anomalies exist

Conclusion: The ADMINISTRATOR table is in BCNF and is properly normalized. The structure supports efficient queries and maintains referential integrity.

4.4 BATCHES Table Normalization

Analysis:

First Normal Form (1NF): ✓ PASS

- All attributes contain only atomic values
- No repeating groups or multivalued attributes
- Each cell contains a single scalar value
- The table is in 1NF

Second Normal Form (2NF): ✓ PASS

- All non-key attributes (`batch_name`, `program`, `year_of_study`, `remarks`) are fully dependent on the primary key (`batch_id`)
- No partial dependencies exist
- The table is in 2NF

Third Normal Form (3NF): ✓ PASS

Functional Dependencies:

- `batch_id` → `batch_name`
- `batch_id` → `program`
- `batch_id` → `year_of_study`
- `batch_id` → `remarks`
- No non-key attribute depends on another non-key attribute
- All dependencies are on the primary key only
- No transitive dependencies exist

Boyce-Codd Normal Form (BCNF): ✓ PASS

- All determinants are candidate keys
- `batch_id` and `batch_name` (unique) are candidate keys
- No anomalies exist

Conclusion: The BATCHES table is in BCNF and is properly normalized.

4.5 STUDENT Table Normalization

Analysis:

First Normal Form (1NF): ✓ PASS

- All attributes (`roll_no`, `name`, `year`, `gender`, `department`, `phone`, `email`, `rm_key`, `status`, `hostel_id`, `batch_id`) contain only atomic values
- No multivalued attributes are present
- No arrays, lists, or nested structures exist
- Each attribute cell contains a single scalar value
- The table is in 1NF

Second Normal Form (2NF): ✓ PASS

- All non-key attributes are fully dependent on the primary key (`student_id`)
- No partial dependencies exist
- `roll_no` is a unique attribute but not part of a composite key; therefore, it does not create partial dependency issues
- Since `student_id` is a single-attribute primary key, all non-key attributes are naturally fully dependent on it
- The table is in 2NF

Third Normal Form (3NF): ✓ PASS with Justification

Functional Dependencies:

- $\text{student_id} \rightarrow \text{roll_no}$ (and $\text{roll_no} \rightarrow \text{student_id}$ due to uniqueness)
- $\text{student_id} \rightarrow \text{name}$
- $\text{student_id} \rightarrow \text{year}$
- $\text{student_id} \rightarrow \text{gender}$
- $\text{student_id} \rightarrow \text{department}$
- $\text{student_id} \rightarrow \text{phone}$
- $\text{student_id} \rightarrow \text{email}$
- $\text{student_id} \rightarrow \text{rm_key}$
- $\text{student_id} \rightarrow \text{status}$
- $\text{student_id} \rightarrow \text{hostel_id}$
- $\text{student_id} \rightarrow \text{batch_id}$
- No non-key attribute depends on another non-key attribute
- Department, year, and other attributes depend directly on student_id , not on each other
- The rm_key attribute serves as a reference to the ROOMMATES table and does not create a transitive dependency
- The hostel_id and batch_id attributes are foreign key references and do not create transitive dependencies
- No chain of dependencies ($A \rightarrow B \rightarrow C$) exists where non-key attributes are involved

Justification: The rm_key , hostel_id , and batch_id attributes are not derived attributes; they are explicit foreign key references that link to other tables. This design pattern is appropriate and does not violate 3NF.

Boyce-Codd Normal Form (BCNF): ✓ PASS

- All determinants are candidate keys
- student_id and roll_no (unique) are candidate keys
- No anomalies exist

Conclusion: The STUDENT table is in BCNF and is properly normalized. The structure supports efficient queries and maintains referential integrity.

4.6 ROOM Table Normalization

Analysis:

First Normal Form (1NF): ✓ PASS

- All attributes (`hostel_id`, `phase`, `wing_code`, `floor_no`, `room_no`, `bed_id`, `allotted_to`) are atomic
- `phase`, `wing_code`, `floor_no`, `room_no`, and `bed_id` are all single scalar values
- No repeating groups or multivalued attributes
- No nested or composite values within cells
- The table is in 1NF

Second Normal Form (2NF): ✓ PASS

- All non-key attributes are fully dependent on the primary key (`room_id`)
- `hostel_id`, `phase`, `wing_code`, `floor_no`, `room_no`, `bed_id`, and `allotted_to` all depend on `room_id`
- No partial dependencies exist
- Since `room_id` is a single-attribute primary key, there are no composite key considerations
- The table is in 2NF

Third Normal Form (3NF): ✓ PASS

Functional Dependencies:

- `room_id` → `hostel_id`
- `room_id` → `phase`
- `room_id` → `wing_code`
- `room_id` → `floor_no`
- `room_id` → `room_no`
- `room_id` → `bed_id`
- `room_id` → `allotted_to`

Potential transitive dependency concern: `phase`, `wing_code`, `floor_no` collectively represent a location within the hostel. However, these attributes are NOT derived from other attributes; they are fundamental structural properties of the room entity.

Analysis: While `phase`, `wing_code`, and `floor_no` together logically identify a location, they do not form a chain of dependencies (`room_id` → X → Y pattern). Each is a direct property of the room.

Normalization decision: Separating these attributes into a separate LOCATIONS table would be over-normalization. It would require additional joins for every query and violate the principle of pragmatic normalization. The current design maintains 3NF because no non-key attribute is dependent on another non-key attribute; all dependencies point directly to `room_id`.

Unique Composite Constraint: The unique constraint on (`hostel_id`, `room_no`, `bed_id`) ensures that no duplicate room+bed combination exists within a hostel, which is a business rule enforcement and does not violate normalization.

Boyce-Codd Normal Form (BCNF): ✓ PASS

- All determinants are candidate keys
- `room_id` is the primary key
- The composite unique key (`hostel_id`, `room_no`, `bed_id`) is also a candidate key
- No anomalies exist

Conclusion: The ROOM table is in BCNF. The design balances normalization principles with practical database efficiency. Further decomposition is not recommended.

4.7 ROOMMATES Table Normalization

Analysis:

First Normal Form (1NF): ✓ PASS

- All attributes (`student1_id`, `student2_id`) are atomic
- `student1_id` and `student2_id` contain single scalar values (foreign key references)
- `student2_id` can be NULL for single occupancy scenarios, which is acceptable
- No repeating groups or multivalued attributes
- The table is in 1NF

Second Normal Form (2NF): ✓ PASS

- Both non-key attributes (`student1_id` and `student2_id`) are fully dependent on the primary key (`rm_key`)
- No partial dependencies exist
- `student1_id` depends on `rm_key` (knowing the roommate group tells us who the first student is)
- `student2_id` depends on `rm_key` (knowing the roommate group tells us who the second student is, if present)
- The table is in 2NF

Third Normal Form (3NF): ✓ PASS

Functional Dependencies:

- `rm_key` → `student1_id`
- `rm_key` → `student2_id`
- No non-key attribute depends on another non-key attribute
- `student1_id` does not determine `student2_id`, and vice versa
- Both attributes are independent and depend only on `rm_key`
- No transitive dependencies or derived attributes exist

Boyce-Codd Normal Form (BCNF): ✓ PASS

- All determinants are candidate keys
- `rm_key` is the only determinant and is the primary key
- No anomalies exist

Conclusion: The ROOMMATES table is in BCNF. The explicit mapping of roommate pairs supports efficient querying and maintains clear semantics.

4.8 ROOM_ALLOC_ZONES Table Normalization

Analysis:

First Normal Form (1NF): ✓ PASS

- All attributes are atomic
- No repeating groups or multivalued attributes
- The table is in 1NF

Second Normal Form (2NF): ✓ PASS

- All non-key attributes are fully dependent on the primary key (`zone_id`)
- No partial dependencies exist
- The table is in 2NF

Third Normal Form (3NF): ✓ PASS

Functional Dependencies:

- `zone_id → hostel_id`
- `zone_id → batch_id`
- `zone_id → start_room_no`
- `zone_id → end_room_no`
- `zone_id → remarks`
- No non-key attribute depends on another non-key attribute
- All dependencies are on the primary key only
- The `hostel_id` and `batch_id` attributes are foreign key references and do not create transitive dependencies
- No transitive dependencies exist

Boyce-Codd Normal Form (BCNF): ✓ PASS

- All determinants are candidate keys

- `zone_id` is the only determinant and is the primary key
- No anomalies exist

Conclusion: The `ROOM_ALLOC_ZONES` table is in BCNF and is properly normalized.

4.9 Summary of Normalization

Table	1NF	2NF	3NF	BCNF	Status
HOSTEL	✓ Pass	✓ Pass	✓ Pass	✓ Pass	BCNF Compliant
ADMINISTRATOR	✓ Pass	✓ Pass	✓ Pass	✓ Pass	BCNF Compliant
BATCHES	✓ Pass	✓ Pass	✓ Pass	✓ Pass	BCNF Compliant
STUDENT	✓ Pass	✓ Pass	✓ Pass	✓ Pass	BCNF Compliant
ROOM	✓ Pass	✓ Pass	✓ Pass	✓ Pass	BCNF Compliant
ROOMMATES	✓ Pass	✓ Pass	✓ Pass	✓ Pass	BCNF Compliant
ROOM_ALLOC_ZONES	✓ Pass	✓ Pass	✓ Pass	✓ Pass	BCNF Compliant

All seven tables conform to Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF), which is the industry standard for transactional database systems. The design provides an excellent balance between data integrity, query performance, and practical usability.

5. Key Design Decisions and Justifications

5.1 Composite Unique Constraint in ROOM Table

Decision: The `ROOM` table includes a unique constraint on `(hostel_id, room_no, bed_id)`.

Justification:

- Ensures no duplicate room+bed combination exists within a hostel
- Supports double occupancy scenarios where `room_no` is the same but `bed_id` differs (A1, B1)
- Enforces business rule that each bed must be uniquely identifiable within a hostel
- Does not violate normalization as it is a candidate key

5.2 Explicit ROOMMATES Table

Decision: A separate `ROOMMATES` table was created rather than storing roommate information directly in the `STUDENTS` table.

Justification:

- Provides a clear, queryable mapping of roommate relationships
- Enables efficient bulk operations on roommate groups
- Maintains referential integrity through explicit foreign keys

- Supports future extensions (e.g., roommate preferences, request histories)
- Eliminates ambiguity in representing single occupancy (NULL vs. self-reference)

5.3 rm_key Default Value in STUDENT Table

Decision: The STUDENTS table includes an `rm_key` attribute with a default value of 0.

Justification:

- 0 serves as a sentinel value indicating "no roommate assignment yet"
- Since MySQL AUTO_INCREMENT typically starts at 1, this value avoids conflicts
- Simplifies queries filtering for unassigned students
- Maintains referential integrity without requiring NULL (which could be ambiguous)

5.4 Detailed Room Identification

Decision: The ROOM table includes `phase`, `wing_code`, and `floor_no` as separate attributes.

Justification:

- Enables queries based on location criteria (e.g., "all rooms on floor 3")
- Supports reporting and analytics on room distribution
- Facilitates user-friendly room identification in the UI/application
- Maintains data for maintenance and administrative purposes
- Does not violate 3NF as discussed in Section 4.6

5.5 VARCHAR Primary Key for BATCHES

Decision: The BATCHES table uses `batch_id` as a VARCHAR(50) primary key rather than an auto-increment integer.

Justification:

- Batch IDs may follow institutional naming conventions (e.g., "2024-UG-CSE", "2023-PhD")
- Provides semantic meaning and human readability
- Simplifies data import and integration with external systems
- Avoids the need for lookup operations to understand batch identity

5.6 Multiple Administrators per Hostel

Decision: The ADMINISTRATOR table does not enforce a unique constraint on `hostel_id`, allowing multiple administrators per hostel.

Justification:

- Supports real-world scenarios where multiple staff members manage a single hostel
- Provides flexibility for administrative hierarchy and workload distribution

- Enables rotation of administrative responsibilities without schema changes
 - Maintains referential integrity through foreign key constraints
-

6. Potential Future Normalization Considerations

6.1 Current Assessment

The current schema is appropriately normalized for the hostel allocation system's current and foreseeable requirements. All tables are in BCNF, which is the highest practical normalization level for production databases. Further normalization to Fourth Normal Form (4NF) or Fifth Normal Form (5NF) is not recommended at this stage.

The schema is production-ready and scalable, with clear pathways for future extensions without requiring structural redesign.

6.2 Future Extension: Department Table

Scenario: If departments become more complex entities with attributes beyond a simple string (e.g., `department_head`, `location`, `phone`), normalization may be beneficial.

Potential Change:

- Create a DEPARTMENTS table: `DEPARTMENTS(dept_id, dept_name, dept_head, location, phone)`
- Modify STUDENT table: `STUDENT(..., dept_id, ...)`

Rationale: This would eliminate potential redundancy if department information is repeatedly stored for multiple students. However, in the current design, storing department as a simple string is pragmatic and avoids unnecessary joins.

6.3 Future Extension: Gender and Status as Lookup Tables

Scenario: If ENUM attributes require metadata (e.g., gender descriptions, status descriptions for reporting), normalization may be beneficial.

Potential Changes:

- `GENDER_TYPES(gender_id, gender_name, description)`
- `STATUS_TYPES(status_id, status_name, description)`

Rationale: This would support multi-language support and detailed reporting. However, for the current system using English ENUM values, this would be over-normalization.

6.4 Future Extension: Audit and History Tables

Scenario: If the system needs to track allocation history or student status changes over time.

Potential Addition:

- `ALLOCATION_HISTORY(history_id, room_id, student_id, allocation_date, deallocation_date, reason)`
- `STUDENT_STATUS_HISTORY(history_id, student_id, previous_status, new_status, change_date, remarks)`

Rationale: These would support compliance, auditing, and analytics. They would not violate current normalization but would be additions to the schema.

6.5 Future Extension: Room Type Table

Scenario: If room types become more complex with additional metadata (e.g., amenities, pricing, capacity).

Potential Change:

- Create a `ROOM_TYPES` table: `ROOM_TYPES(room_type_id, type_name, capacity, amenities, price)`
- Modify `ROOM` table: `ROOM(..., room_type_id, ...)`

Rationale: This would support detailed room classification and dynamic room type management. However, the current design adequately handles single and double occupancy through the `HOSTEL` table's `occupancy_type` attribute.

6.6 Recommendation

The current design requires **no immediate further normalization**. The BCNF schema is optimal for:

- Production system implementation
- Query performance
- Data integrity maintenance
- Future scalability

However, development teams should monitor for the scenarios outlined above and re-evaluate normalization if system requirements evolve significantly.

7. Referential Integrity and Constraints

7.1 Foreign Key Relationships

The database maintains referential integrity through the following foreign key relationships:

1. **ADMINISTRATOR** → **HOSTEL**: Each administrator must be assigned to an existing hostel
 - Cascade behavior: `ON UPDATE CASCADE, ON DELETE RESTRICT`
 - Ensures administrators are not left orphaned if hostel information changes

2. **STUDENT** → **HOSTEL**: Each student must be assigned to an existing hostel
 - Cascade behavior: ON UPDATE CASCADE, ON DELETE CASCADE
 - If a hostel is deleted, associated students are also removed
3. **STUDENT** → **BATCHES**: Each student may belong to a batch (optional)
 - Cascade behavior: ON UPDATE CASCADE, ON DELETE SET NULL
 - If a batch is deleted, students' batch_id is set to NULL
4. **ROOM** → **HOSTEL**: Each room must belong to an existing hostel
 - Cascade behavior: ON UPDATE CASCADE, ON DELETE CASCADE
 - Ensures rooms are not left orphaned if hostel is deleted
5. **ROOM** → **STUDENT**: Each room can be allocated to a student (optional)
 - Cascade behavior: ON UPDATE CASCADE, ON DELETE SET NULL
 - If a student is deleted, the room's allotted_to is set to NULL (room becomes unoccupied)
6. **ROOMMATES** → **STUDENT**: Both student references must exist
 - Cascade behavior: ON UPDATE CASCADE, ON DELETE CASCADE
 - Ensures roommate pairs are deleted if either student is removed
7. **ROOM_ALLOC_ZONES** → **HOSTEL**: Each zone must belong to an existing hostel
 - Cascade behavior: ON UPDATE CASCADE, ON DELETE CASCADE
8. **ROOM_ALLOC_ZONES** → **BATCHES**: Each zone must be assigned to an existing batch
 - Cascade behavior: ON UPDATE CASCADE, ON DELETE CASCADE

7.2 Unique Constraints

- **HOSTEL.hostel_name**: Ensures no duplicate hostel names
- **ADMINISTRATOR.email**: Ensures each administrator has a unique email address
- **BATCHES.batch_name**: Ensures no duplicate batch names
- **STUDENT.roll_no**: Ensures each student has a unique roll number
- **ROOM(hostel_id, room_no, bed_id)**: Ensures no duplicate room+bed combination within a hostel

7.3 Default Values

- **STUDENT.rm_key**: Default value of 0 indicates no roommate assignment
 - **STUDENT.status**: Default value of 'Active' for new student records
 - **ROOM.bed_id**: Default value of 'A1' for single occupancy or first bed in double occupancy
 - **BATCHES.program**: Default value of 'UG' for undergraduate program
-

8. Conclusion

The IIT Jammu Hostel Allocation System database is designed with a well-normalized schema comprising seven tables: HOSTEL, ADMINISTRATOR, BATCHES, STUDENT, ROOM, ROOMMATES, and ROOM_ALLOC_ZONES. All tables conform to Boyce-Codd Normal Form (BCNF), providing an optimal balance between data integrity, query efficiency, and practical usability.

The design decisions reflect industry best practices:

- Surrogate keys for simplified relationships
- Explicit mapping tables for many-to-many scenarios
- Atomic attributes with appropriate data types
- Referential integrity through cascading foreign keys
- Comprehensive attribute sets supporting all operational requirements
- Unique constraints enforcing business rules

The schema is production-ready and scalable, with clear pathways for future extensions without requiring structural redesign. The normalization analysis confirms that the current design eliminates redundancy, maintains data integrity, and provides a solid foundation for reliable hostel allocation operations.

The embedded E-R diagram provides comprehensive conceptual understanding of the database design, offering both academic and practical perspectives on the system architecture.

Appendix: SQL Schema Creation Script


```

SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE IF EXISTS room_alloc_zones;
DROP TABLE IF EXISTS batches;
DROP TABLE IF EXISTS roommates;
DROP TABLE IF EXISTS room;
DROP TABLE IF EXISTS student;
DROP TABLE IF EXISTS administrator;
DROP TABLE IF EXISTS hostel;

SET FOREIGN_KEY_CHECKS = 1;

CREATE DATABASE IF NOT EXISTS iitjammu_hostel;
USE iitjammu_hostel;

-- HOSTEL Table
CREATE TABLE hostel (
    hostel_id INT AUTO_INCREMENT PRIMARY KEY,
    hostel_name VARCHAR(50) UNIQUE NOT NULL,
    gender ENUM('Male', 'Female', 'Other'),
    year_allocated INT NOT NULL,
    occupancy_type ENUM('Single', 'Double') NOT NULL,
    total_rooms INT
);

-- ADMINISTRATOR Table
CREATE TABLE administrator (
    admin_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    designation VARCHAR(100) NOT NULL,
    contact_phone VARCHAR(15),
    email VARCHAR(100) UNIQUE,
    hostel_id INT NOT NULL,
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

-- BATCHES Table
CREATE TABLE batches (
    batch_id VARCHAR(50) PRIMARY KEY,
    batch_name VARCHAR(50) UNIQUE NOT NULL,
    program ENUM('UG', 'MTech', 'PhD', 'Guest', 'Other') DEFAULT 'UG',

```

```

        year_of_study INT NULL,
        remarks VARCHAR(100)
    );

-- STUDENT Table
CREATE TABLE student (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    roll_no VARCHAR(15) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    year INT NOT NULL,
    gender ENUM('Male', 'Female', 'Other') NOT NULL,
    department VARCHAR(50),
    phone VARCHAR(15),
    email VARCHAR(100),
    rm_key INT DEFAULT 0,
    status ENUM('Active', 'Graduated', 'Left') DEFAULT 'Active',
    hostel_id INT NOT NULL,
    batch_id VARCHAR(50),
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (batch_id) REFERENCES batches(batch_id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

-- ROOM Table
CREATE TABLE room (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    hostel_id INT NOT NULL,
    phase VARCHAR(5),
    wing_code VARCHAR(10),
    floor_no INT,
    room_no VARCHAR(10) NOT NULL,
    bed_id ENUM('A1', 'B1') DEFAULT 'A1',
    allotted_to INT NULL,
    UNIQUE(hostel_id, room_no, bed_id),
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (allotted_to) REFERENCES student(student_id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

-- ROOMMATES Table
CREATE TABLE roommates (
    rm_key INT PRIMARY KEY,

```

```
student1_id INT NOT NULL,  
student2_id INT NULL,  
FOREIGN KEY (student1_id) REFERENCES student(student_id)  
    ON UPDATE CASCADE ON DELETE CASCADE,  
FOREIGN KEY (student2_id) REFERENCES student(student_id)  
    ON UPDATE CASCADE ON DELETE SET NULL  
);  
  
-- ROOM_ALLOC_ZONES Table  
CREATE TABLE room_alloc_zones (  
    zone_id INT AUTO_INCREMENT PRIMARY KEY,  
    hostel_id INT NOT NULL,  
    batch_id VARCHAR(50) NOT NULL,  
    start_room_no VARCHAR(10) NOT NULL,  
    end_room_no VARCHAR(10) NOT NULL,  
    remarks VARCHAR(100),  
    FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)  
        ON UPDATE CASCADE ON DELETE CASCADE,  
    FOREIGN KEY (batch_id) REFERENCES batches(batch_id)  
        ON UPDATE CASCADE ON DELETE CASCADE  
);
```

End of Report